pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

What an Agent Is

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: what an agent is

The week's bond (synthetic)

Coupon	5.000%, semiannual, 30/360
Settles	2026-10-02
Matures	2031-10-02
Yield	5.060%
Saved run	recorded 2026-10-08

Today

1. A model, tools, a loop
2. The key, kept out of Git
3. A request on the wire, and replay
4. A tool is a function plus a description
5. The first tool request, by hand
6. The tool run, the result back
7. Reading the answer; what it cost

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for agents

What an agent produces: text, every number from a tool

Allowed names

the agent returned

the saved run

the brief states

Never called

the agent's opinion or view

what the agent believes

the agent's advice

On the synthetic book (from day 3): a description, never a recommendation

Allowed names

DV01 per 100 of face

position DV01 in dollars

the largest DTS contributions in the file

Never called

cheap, rich, or attractive

a trade idea or a recommendation

overweight or underweight

A model, tools, a loop

The model

runs on the provider's servers

receives messages and tools

returns text, or tool requests

runs nothing on your machine

The tools

runs on your machine

your functions, described

run only when your code runs them

today: bondmath's dv01

The loop

runs on your code

send, run tools, send back

stop at text or a step limit

today: two requests by hand

Only the model is remote. Every number a tool returns is computed by your own code.

The key and the mode

get_secret looks in order

1. the environment
2. Colab's Secrets panel
3. my_bondmath\.env

Before anything is sent

```
git check-ignore -v .env cache/agents/
```

```
.gitignore:4:.env    .env
.gitignore:5:cache/    cache/agents/
```

```
OpenAI key loaded: False. Mode: replay.
```

The key cell is the only cell that touches the key, and it prints only whether one loaded.

A request on the wire

```
POST /v1/chat/completions
Authorization: (the key, a header)
{
  "model": "gpt-4.1-mini",
  "messages": [...], "tools": [...],
  "temperature": 0, "seed": 42, ...
}
```

exchange_key: method, path, body

```
"the DV01 ..."   1d42e20fa5c722d7...

"the dv01 ..."   6e361ffc2a182cbf...
```

keys sorted, no spaces; never a header

4 saved exchanges copied for day 1

replay

no key: the saved response, or LookupError

live

your key: sent, then saved under its key

Each request carries every reply before it: a saved run fixes the whole path.

A tool is a function plus a description

```
def dv01(settlement: str, maturity: str,
         coupon_pct: float, yield_pct: float):
    """The DV01 in price points per 100 of
    face: the price change for a 1 bp fall.
    Args:
        coupon_pct: the annual coupon in
            percent, 5.0 for a 5% coupon.
```

What the model receives

name	the function's name: dv01
description	the docstring's first paragraph
parameters	type hints + Args: lines
required	every parameter
strict	true

`dv01("2026-10-02", "2031-10-02", 5.0, 5.06) returns {'dv01_per_100': 0.043625}`

The docstring is the prompt: the model reads it, and nothing else of your code.

The first tool request

Sent

system: DESK_PROMPT

user: the question, with its date

tools: [dv01]

parallel_tool_calls: false

Returned in the saved run

finish_reason	tool_calls
content	None: no text at all
name	dv01
id	call_ and 24 more characters

```
arguments the model wrote: {"settlement":"2026-10-02", "maturity":"2031-10-02", "coupon_pct":5, "yield_pct":5.06}
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 1 request, 432 prompt and 42 completion tokens; saved exchange recorded 2026-10-08; replayed (no key loaded).

Nothing has run yet. The DV01 does not exist until your code runs the tool.

Running the tool, and the result back

1. Run it

dv01(**json.loads(arguments))

2. tool_message

the id, and the result as JSON

3. Send again

reply and result added

What the agent did, step by step

```
1  assistant  dv01, the arguments; tool_calls
1  tool  {'dv01_per_100': 0.043625}
2  assistant  the text; stop
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 2 requests, 952 prompt and 124 completion tokens; saved run recorded 2026-10-08; replayed (no key loaded).

The model holds nothing between requests: each one carries the whole conversation.

Reading the answer

13
numbers in the saved answer

12
written in the question

1
the tool's value

0
from neither

A regular expression on free text: a first check with two gaps

It cannot say which tool request a number came from.

A different rounding of another value could match by chance.

Day 4: every figure names its tool request, and check_figures rejects one that does not trace.

What it cost

432 / 42

first request: prompt / completion

520 / 82

second request: prompt / completion

US$0.000579

the run at the list price

gpt-4.1-mini, standard tier, per million tokens

US$0.40 prompt, US$1.60 completion; usage_cost (Course 7) on each request, added

1,000 runs like it: US$0.58

The second request resends everything the first held: every step costs more prompt tokens.

Source, read 8 October 2026: developers.openai.com/api/docs/pricing

Excel and Git

Excel side by side (tested in Excel)

`=PRICE(DATE(2026,10,2),DATE(2031,10,2),0.05,0.0506,100,2,0)`	99.73784463676976
`=MDURATION(DATE(2026,10,2),DATE(2031,10,2),0.05,0.0506,2,0)`	4.373954476118345
`=B3*(B2+B4)/10000  (DV01 per 100)`	0.043624879198739516
`=B2*C2+B3*C3  (the run's cost)`	0.0005792
`=SUMPRODUCT(B2:B3,D2:D3)  (prices as text)`	0

```
!! __pycache__/
!! cache/
```

One commit: four earlier modules beside agentkit, 7 tests. cache/ marked !!, out of Git.

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

Units, conventions, and what the function must never do: what you asked for.

2

Code back

Written for the course in the style of an AI answer. Read it.

3

Test first

Assert what the docstring promises, on a fake server. Run it: it fails.

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the Args: line says a decimal, so the model sends 0.05 and gets 0.038041.

Code that runs is not the same as code that is right.

What you can do now

- Say what an agent is: model, tools, loop
- Keep the key in .env, out of Git, never printed
- Turn a function into a tool: tool_schema
- Send a tool request by hand, send the result back
- Trace an answer's numbers; price the run

Week 1, Day 2 The agent loop by hand